

统计机器学习基础

Foundations of Statistical Machine Learning

——聚类算法

Kai Chen

kaichen6@csu.edu.cn

24th September 2025



中南大学
CENTRAL SOUTH UNIVERSITY

数学与统计学院
SCHOOL OF MATHEMATICS AND STATISTICS

- 1 聚类理论
 - 聚类的定义
 - 距离度量
- 2 聚类算法
 - 层次算法
 - 分区算法
 - K-means 算法
 - 聚类的影响因素
- 3 补充学习

- 1 聚类理论
 - 聚类的定义
 - 距离度量
- 2 聚类算法
- 3 补充学习

什么是聚类？



- 它们之间是否存在“分组”？
- 每个组是什么？
- 有多少个？
- 如何识别它们？

什么是聚类？

- 聚类：将一组对象划分为相似对象类别的过程
 - 高类内相似性
 - 低类间相似性
 - 这是无监督学习最常见的形式
- 无监督学习 = 从原始（无标签、无注释等）数据中学习，与监督学习相对，后者给出了示例的分类
- 这是一项常见且重要的任务，在科学、工程、信息科学等领域有大量应用
 - 将执行相同功能的基因分组
 - 将具有相似政治观点的个人分组
 - 对相似主题的文档进行分类
 - 从图片中识别相似对象

示例

- People



- Images



- Language

Piotr *Pyotr* *Petros* *Pietro* *Pedro* *Pierre* *Piero* *Peter* *Peder* *Peka* *Peadar*

- species



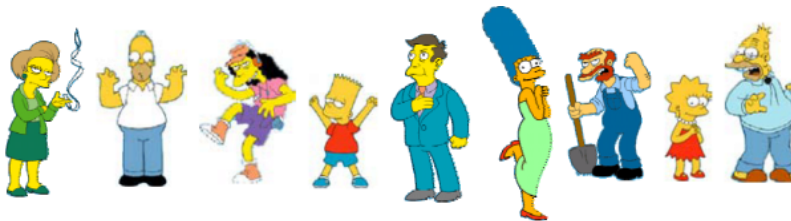
聚类面临的问题

- 这些对象之间的自然分组是什么？——“组性”的定义
- 什么使对象“相关”？——“相似性/距离”的定义
- 对象的表示——向量空间？归一化？
- 多少个聚类？——先验固定？或是完全数据驱动（避免聚类过大或过小）？
- 聚类算法
 - 分区算法
 - 层次算法
- 形式基础和收敛

”形式基础”指的是聚类算法的理论支撑，包括算法的数学模型、假设条件、以及算法如何根据这些假设进行操作。

”收敛”在聚类算法中通常指的是算法在迭代过程中逐渐接近一个稳定状态，即算法的输出不再显著变化。

自然分组



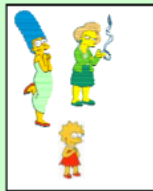
Clustering is subjective



Simpson's Family



School Employees



Females



Males

© Eric

分组是主观的

什么是相似性？



难以定义！但我们看到它时就知道它是什么

- 相似性的真正含义是一个哲学问题。我们将采取更实用的方法
- 取决于表示和算法。对于许多表示/算法，更容易从向量之间的距离（而不是相似性）来思考

- 1 聚类理论
 - 聚类的定义
 - 距离度量
- 2 聚类算法
- 3 补充学习

距离度量的性质

- $D(A, B) = D(B, A)$ 对称性
- $D(A, A) = 0$ 自相似性的常数性
- $D(A, B) = 0$ 当且仅当 $A = B$ 正定性分离
- $D(A, B) \leq D(A, C) + D(B, C)$ 三角不等式

理想距离度量性质背后的直觉

- $D(A, B) = D(B, A)$ 对称性
否则你可以声称 “Alex 像 Bob, 但 Bob 一点也不像 Alex”
- $D(A, A) = 0$ 自相似性常数
否则你可以声称 “Alex 比 Bob 更像 Bob”
- $D(A, B) = 0$ 当且仅当 $A = B$ 正分离
否则你的世界中有不同但你无法区分的对象
- $D(A, B) \leq D(A, C) + D(B, C)$ 三角不等式
否则你可以声称 “Alex 非常像 Bob, Alex 非常像 Carl, 但 Bob 非常不像 Carl”

距离度量: Minkowski 度量

- 假设两个对象 x 和 y 都有 p 个特征

$$x = (x_1, x_2, \dots, x_p)$$

$$y = (y_1, y_2, \dots, y_p)$$

- Minkowski 度量定义为

$$d(x, y) = \sqrt[r]{\sum_{i=1}^p |x_i - y_i|^r}$$

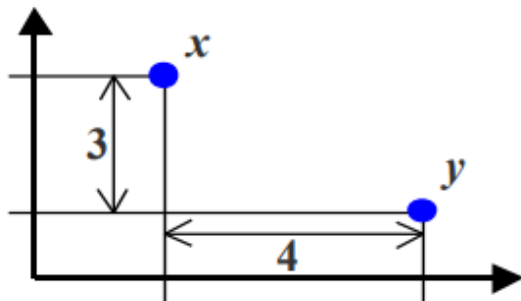
- 最常见的闵可夫斯基度量

1. $r = 2$ 欧几里得距离 $d(x, y) = \sqrt{\sum_{i=1}^p |x_i - y_i|^2}$

2. $r = 1$ 曼哈顿距离 $d(x, y) = \sum_{i=1}^p |x_i - y_i|$

3. $r = +\infty$ 上确界距离 $d(x, y) = \max_i |x_i - y_i|$

距离度量：举例



曼哈顿距离: $4 + 3 = 7$

欧几里得距离: $\sqrt{4^2 + 3^2} = 5$

上确界距离: $\max\{4, 3\} = 4$

距离度量：汉明距离

汉明距离是指两个等长字符串之间，在相同位置上不同字符的个数。

汉明距离 d_H 的计算公式为：

$$d_H(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^n |a_i - b_i|$$

其中， $|a_i - b_i|$ 表示在第 i 个位置上两个向量的差异。如果 a_i 和 b_i 相同（即 $a_i = b_i$ ），则 $|a_i - b_i| = 0$ ；如果不同，则 $|a_i - b_i| = 1$ 。

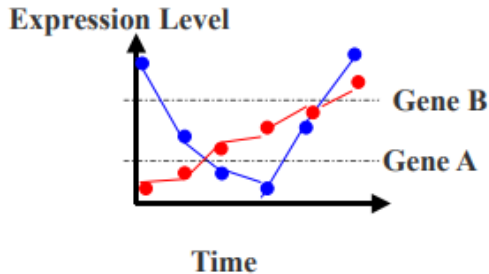
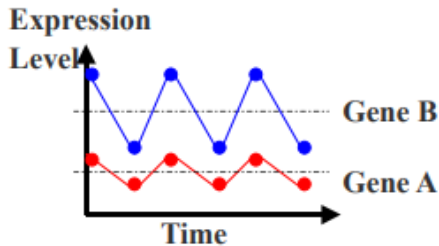
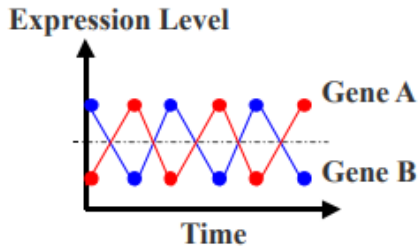
当所有特征都是二元时（即特征只能取 0 或 1 的值），曼哈顿距离称为汉明距离

例：17 个条件下的基因表达水平（1-高，0-低）

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
GeneA	0	1	1	0	0	1	0	0	1	0	0	1	1	1	0	0	1
GeneB	0	1	1	1	0	0	0	0	1	1	1	1	1	1	0	1	1

Hamming Distance: $\#(01) + \#(10) = 4 + 1 = 5$.

相似性度量：相关系数



相似性度量：相关系数

- 皮尔逊相关系数：皮尔逊相关系数是两个变量的协方差除以它们的标准差的乘积

$$\rho_{X,Y} = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y}$$

对于一个样本，即给定配对数据 $\{(x_1, y_1), \dots, (x_p, y_p)\}$

$$s(x, y) = \frac{\sum_{i=1}^p (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^p (x_i - \bar{x})^2 \times \sum_{i=1}^p (y_i - \bar{y})^2}}$$
$$|s(x, y)| \leq 1$$

其中 $\bar{x} = \frac{1}{p} \sum_{i=1}^p x_i$, $\bar{y} = \frac{1}{p} \sum_{i=1}^p y_i$ 。

- 特殊情况：余弦距离

$$s(x, y) = \frac{\vec{x} \cdot \vec{y}}{|\vec{x}| \cdot |\vec{y}|}$$

编辑距离：衡量相似性的通用技术

- 为了衡量两个对象之间的相似性，将其中一个对象转换为另一个对象，并衡量所需的工作量。工作量的度量成为距离，这称为编辑距离或转换距离。

帕蒂和塞尔玛之间的距离。

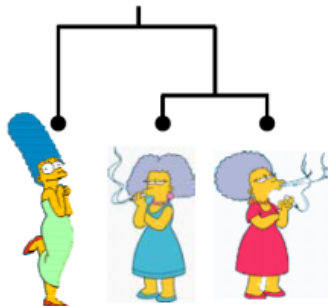
- 更换衣服颜色，1 分
- 更换耳环形状，1 分
- 更换发型，1 分

$$D(\text{帕蒂}, \text{塞尔玛}) = 3$$

玛吉和塞尔玛之间的距离。

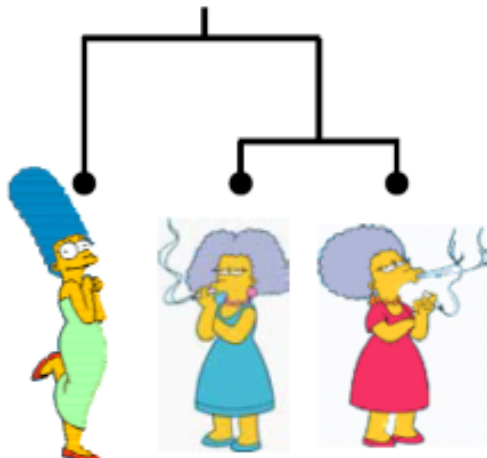
- 更换衣服颜色，1 分
- 增加耳环，1 分
- 降低身高，1 分
- 开始吸烟，1 分
- 减肥，1 分

$$DP(\text{玛吉}, \text{塞尔玛}) = 5$$



聚类算法

- 分区算法：如 K 均值聚类、基于混合模型的聚类
 - 通常从随机（部分）划分开始
 - 迭代改进
- 层次算法
 - 自底向上，聚合
每次迭代合并最相似的组
 - 自顶向下，分裂
每次迭代分裂最不相似的组



Outline

1 聚类理论

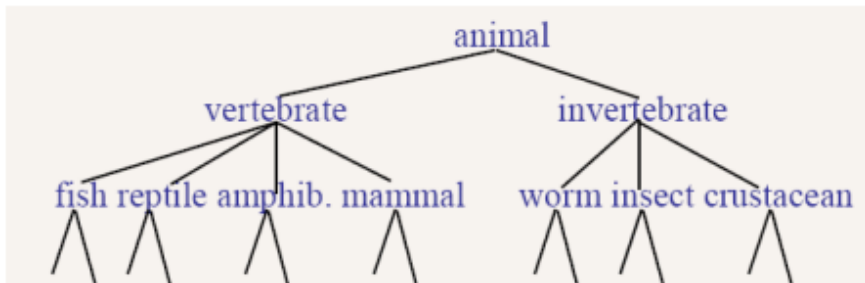
2 聚类算法

- 层次算法
- 分区算法
- K-means 算法
- 聚类的影响因素

3 补充学习

层次聚类

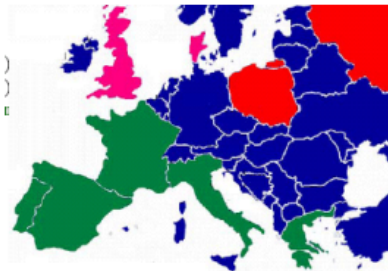
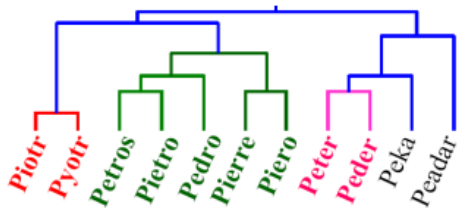
- 从一组对象构建基于树的层次分类（树状图）



- 层次结构通常用于组织信息，例如在网页中
层次结构通过将信息分解成不同层级，帮助用户逐步深入理解复杂的概念或数据集。
- 专注于自动创建数据中的层次结构
利用算法自动识别和构建数据的内在层次关系

树状图

- 相似性度量
评估对象之间的相似程度
在树状图（也称为树形图或层次聚类图）中，两个对象之间的相似性可以通过它们共享的最低内部节点（即最近共同祖先）的高度来表示。
高度越低，表示两个对象越相似。
- 在树状图的期望高度处进行切割来获得聚类结果
每个连通的分支（或分量）形成一个独立的聚类。



层次聚类的分类

- 自底向上聚合聚类

- 从每个对象在单独聚类中开始
- 然后反复合并最接近的聚类对
- 直到只剩下一个聚类

合并的历史形成二叉树或层次结构

- 自顶向下分裂

- 从所有数据在单个聚类中开始
- 考虑将聚类划分为两个的所有可能方式，选择最佳划分
- 并递归地对两者进行操作

最接近的聚类对

两个聚类之间的距离可以通过不同的方法来定义，具体如下：

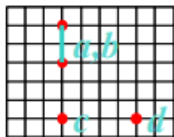
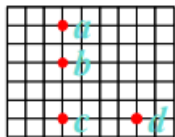
- **单链接（最近邻）**：两个聚类中距离最近的成员之间的距离。
- **全链接（最远邻）**：两个聚类中距离最远的成员之间的距离。
- **质心链接**：两个聚类的质心（重心）之间的距离，通常使用余弦相似度来衡量。
- **平均链接**：计算两个聚类中所有成员对之间的平均距离。

聚类对：单链接方法

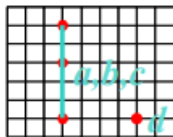
单链接聚类，也称为最近邻聚类，两个组 G 和 H 之间的距离定义为每组中两个最近成员之间的距离：

$$d_{SL}(G, H) = \min_{i \in G, j \in H} d_{i,j}$$

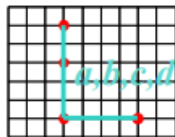
欧几里得距离 $\rightarrow$ 距离矩阵



(1)



(2)



(3)

	b	c	d
a	2	5	6
b		3	5
c			4

	b	c	d
a	2	5	6
b		3	5
c			4

	c	d
a, b	3	5
c		4

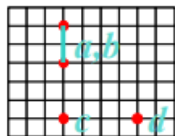
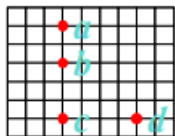
	d
a, b, c	4

聚类对：全链接方法

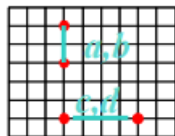
全链接聚类，也称为最远邻聚类，两个组之间的距离定义为两个最远对之间的距离：

$$d_{CL}(G, H) = \max_{i \in G, j \in H} d_{i,j}$$

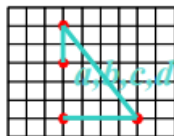
欧几里得距离 $\rightarrow$ 距离矩阵



(1)



(2)



(3)

	<i>b</i>	<i>c</i>	<i>d</i>
<i>a</i>	2	5	6
<i>b</i>		3	5
<i>c</i>			4

	<i>b</i>	<i>c</i>	<i>d</i>
<i>a</i>	2	5	6
<i>b</i>		3	5
<i>c</i>			4

	<i>c</i>	<i>d</i>
<i>a,b</i>	5	6
<i>c</i>		4

	<i>c,d</i>
<i>a,b</i>	6

单链接 vs. 全链接

左边是单链接，右边是全链接

- 单链接

使用单链接聚类构建的树是数据的最小生成树，它以一种最小化边权重（距离）之和的方式连接所有对象。

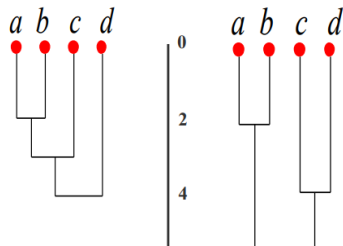
一旦两个簇合并，它们就不会再被考虑，因此我们不会创建循环。

$O(N^2)$

- 全链接

紧凑性属性的簇：组内的所有观察值应该彼此相似。

倾向于产生具有小直径的紧凑簇。



聚类对：平均链接

在实践中，首选的方法是平均链接聚类，它测量所有对之间的平均距离：

$$d_{avg}(G, H) = \frac{1}{n_G n_H} \sum_{i \in G} \sum_{i' \in H} d_{i,i'}$$

其中 n_G 和 n_H 是组 G 和 H 中元素的数量。平均链接聚类代表了单链接和全链接聚类之间的折衷。

聚类对：质心链接

质心可以代表整个聚类的位置或中心，因此质心之间的距离可以作为聚类间距离的代理。

设 C_1 和 C_2 是两个聚类，它们的质心分别为 μ_1 和 μ_2 。质心是聚类中所有点的平均位置。如果我们在欧几里得空间中工作，质心 μ 的计算公式为：

$$\mu = \frac{1}{|C|} \sum_{x \in C} x$$

其中 $|C|$ 是聚类 C 中点的数量。

两个聚类 C_1 和 C_2 之间的距离可以通过它们质心 μ_1 和 μ_2 之间的欧氏距离来衡量：

$$d(C_1, C_2) = \|\mu_1 - \mu_2\|$$

这里， $\|\cdot\|$ 表示欧氏范数。

HAC 的计算复杂度

- 在第一次迭代中，所有 HAC 方法需要计算所有 n 个个体实例对的相似性，为 $O(n^2)$
- 在随后的每个 $n-2$ 次合并迭代中，计算最近创建的聚类与所有其他现有聚类之间的距离
- 为了保持整体 $O(n^2)$ 性能，计算任意两个簇之间的相似度必须在常数时间内完成
例如使用最小生成树或其他高效的相似度计算方法。

- 否则，如果以朴素方式进行，时间复杂度可能是 $O(n^2 \log n)$ 或 $O(n^3)$ 。
“朴素方式”指的是不使用任何优化或高级数据结构，直接按照最直接的方法来计算。

例如，如果每次合并聚类后都重新计算所有聚类对之间的距离，那么随着聚类数量的减少，每次计算的距离对数量会减少，但每个距离计算可能变得更加复杂，因为聚类包含的点更多。

Outline

1 聚类理论

2 聚类算法

- 层次算法
- 分区算法
- K-means 算法
- 聚类的影响因素

3 补充学习

分区聚类

- 分区方法：将 n 个对象划分为 K 个聚类的集合
- 给定：一组对象和簇的数量 K 。
- 目标：找到一个优化所选分区标准的 K 个簇的分区。个聚类的划分，优化所选的划分准则
 - 全局最优：穷举所有可能的分区。
 - 有效的启发式方法：K-means 和 K-medoids 算法。

1 聚类理论

2 聚类算法

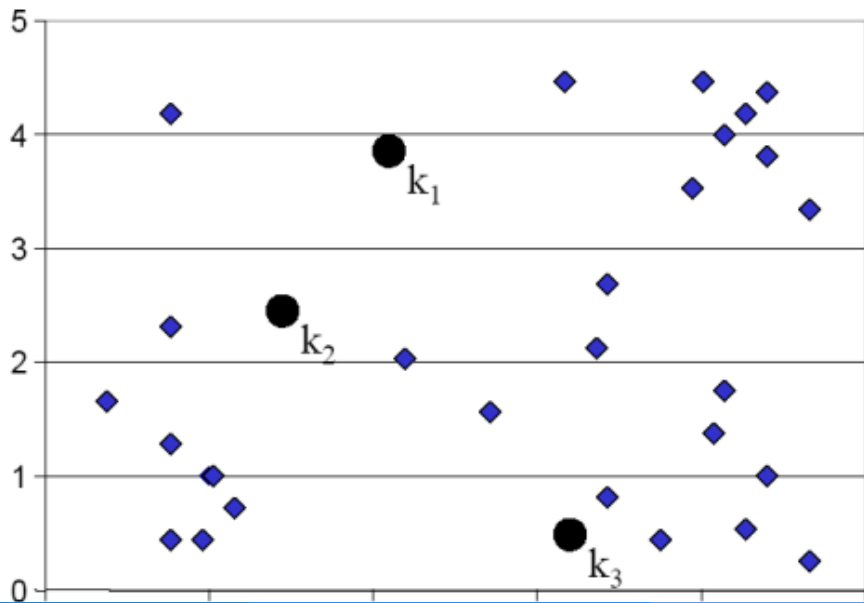
- 层次算法
- 分区算法
- **K-means 算法**
- 聚类的影响因素

3 补充学习

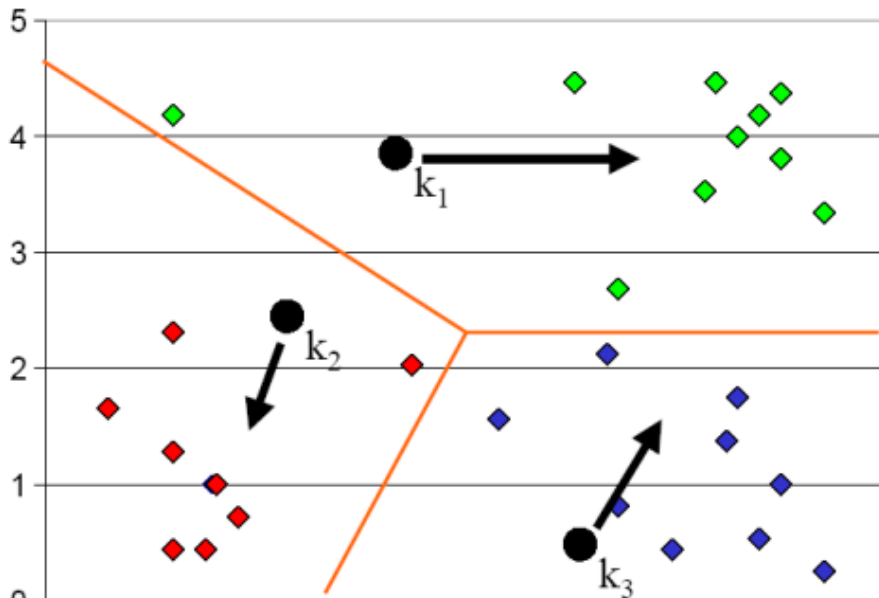
K 均值算法

- ① 确定 k 的值
- ② 必要时随机初始化 k 个聚类中心
- ③ 通过将 N 个对象分配到最近的聚类质心（即重心或均值）来决定它们的类成员资格
- ④ 假设上述找到的隶属关系正确，重新估计 k 个聚类中心
- ⑤ 如果在最后一次迭代中没有 N 个对象改变成员资格，则退出，否则转到步骤 3

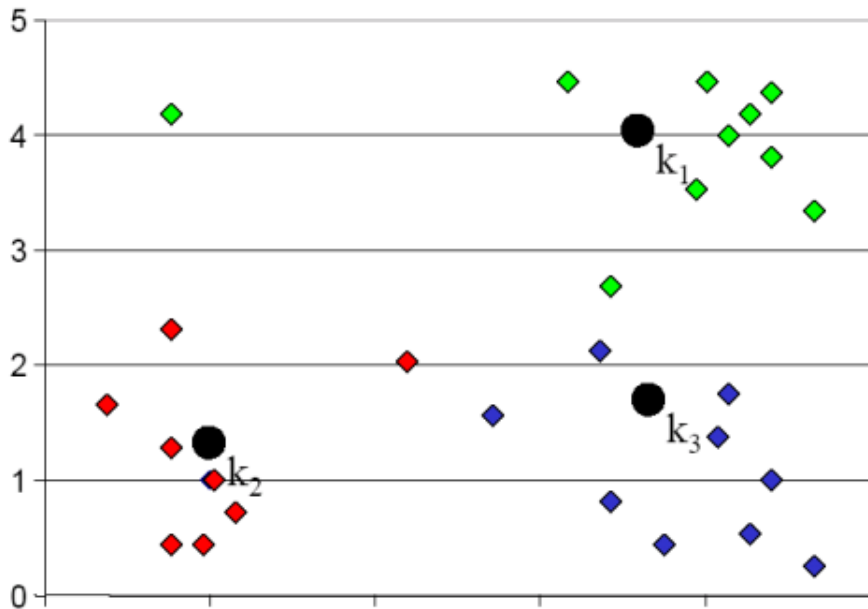
K 均值聚类：步骤 1



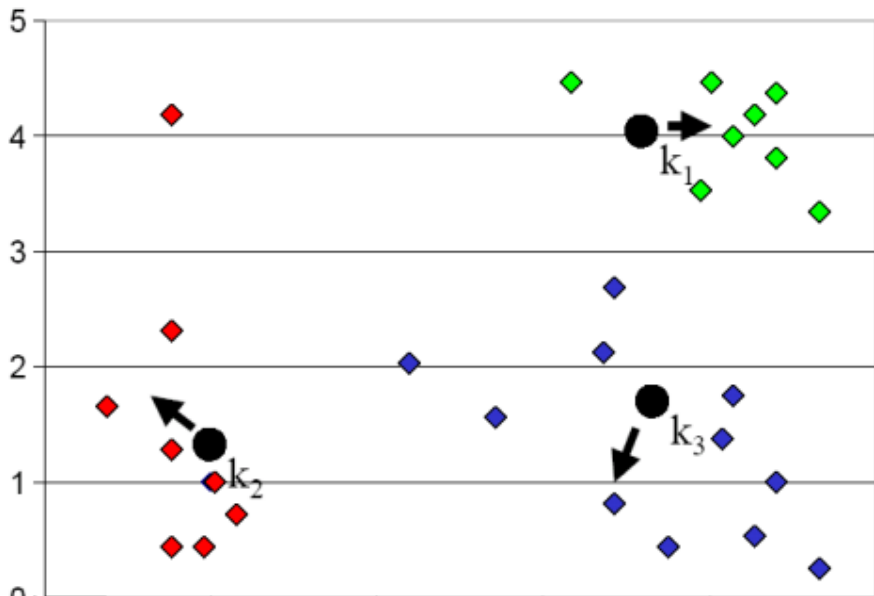
K 均值聚类：步骤 2



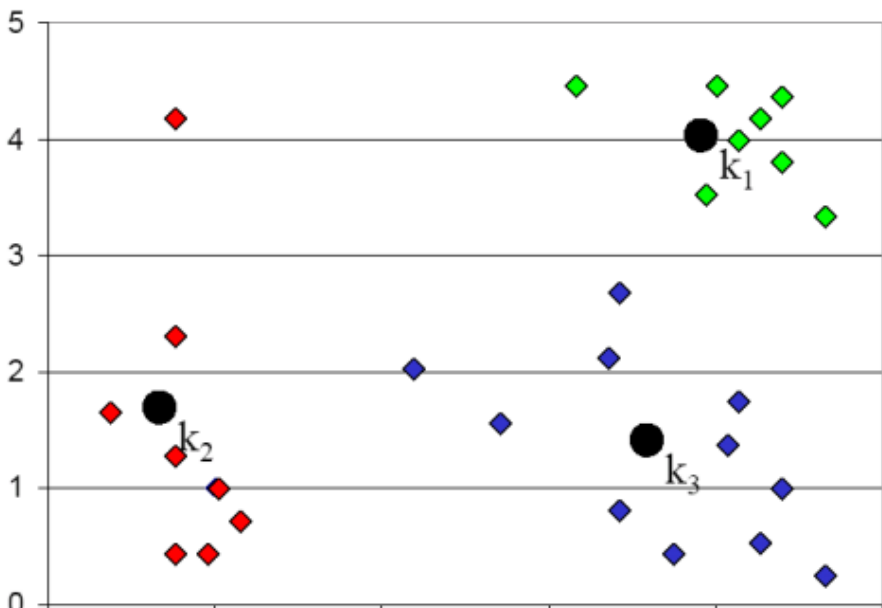
K 均值聚类：步骤 3



K 均值聚类：步骤 4



K 均值聚类：步骤 5

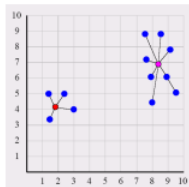


在 K-均值中，我们可以将簇中心视为模型参数，将数据点分配给簇的过程视为隐变量的估计。这样，K-均值算法可以看作是 EM 算法的一个特例

- E-step: 在 K-均值中，E-step 对应于分配步骤，即根据当前簇中心估计每个数据点属于哪个簇。
- M-step: 在 K-均值中，M-step 对应于更新步骤，即根据数据点的分配更新簇中心。

收敛

- K 均值算法为什么会达到固定点？——聚类不再改变的状态
- K 均值是期望最大化 (EM) 算法的一般程序的特殊情况
 - EM 已知会收敛
 - 迭代次数可能很大
- 优度量度量 (评估模型性能的指标)
 - 聚类质心的距离平方和



$$SD_{K_i} = \sum_{j=1}^{m_k} \|x_{ij} - \mu_i\|^2, \quad SD_K = \sum_{i=1}^k SD_{K_i}$$

- 因为每个向量都分配给最近的质心，因此重新分配会单调地降低 SD，从而提高聚类的质量。

时间复杂度

- 计算两个对象之间的距离的时间复杂度为 $O(m)$ ，其中 m 表示数据的维度。
- 分配聚类：需要计算 $O(Kn)$ 次距离，或者在考虑维度的情况下，需要 $O(Knm)$ 次距离计算，这里 K 是簇的数量， n 是数据点的数量。
- 计算质心：在每次迭代中，每个数据点都会被恰好分配到一个质心一次。
- 假设在每次迭代中，上述两个步骤都被执行一次，并且整个过程重复 l 次迭代，则总体时间复杂度为 $O(lKnm)$ 。

Outline

1 聚类理论

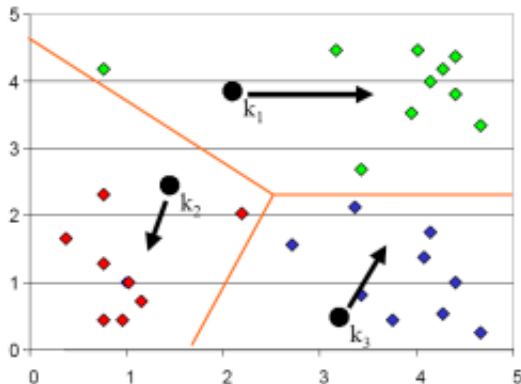
2 聚类算法

- 层次算法
- 分区算法
- K-means 算法
- 聚类的影响因素

3 补充学习

种子选择

- 结果可能因随机种子选择而异



- 某些种子可能导致较差的收敛速度，或收敛到次优聚类
- 使用启发式方法选择好的种子（例如，选择与任何现有均值最不相似的点）
- 尝试多个起点（非常重要!!!）
- 用另一种方法的结果初始化

聚类数量

- 确定聚类的数量 K 。例如，将 n 个数据点划分为预定数量的聚类。
- 找到“正确”的聚类数量是问题的一部分
给定对象，划分为“适当”数量的子集
例如，在查询结果中，理想的 K 值可能事先并不知道，尽管用户界面 (UI) 可能会施加一些限制。
- 解决优化问题：惩罚拥有大量聚类
在应用依赖中很常见，例如，搜索结果的压缩摘要
信息论方法和基于模型的方法可以用于解决这类问题
- 权衡：拥有更多聚类可以使得每个聚类内的数据点更加集中，但同时也要避免聚类数量过多。
- 非参数贝叶斯推断
可以在没有明确参数假设的情况下进行概率推断。

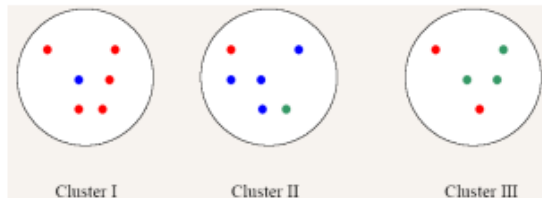
- 内部准则：好的聚类将产生高质量的聚类，其中：
 - 类内（即聚类内）相似性高
 - 类间相似性低
 - 聚类的质量度量取决于对象表示和使用的相似性度量
- 聚类质量的外部标准
 - 通过发现某些或所有隐藏模式或黄金标准数据中的潜在类来度量质量
 - 根据真实情况评估聚类
- 示例：
 - 纯度
 - 聚类中类的熵（或类和聚类之间的互信息）

纯度

- 纯度，聚类中主导类与聚类大小的比率
假设文档具有 C 个黄金标准类，而我们的聚类算法产生 K 个聚类 $\omega_1, \omega_2, \dots, \omega_K$ ，成员数为 n_i

$$Purity(w_i) = \frac{1}{n_i} \max_j (n_{ij}) \quad j \in C$$

- 示例



- 聚类 I: 纯度 = $5/6$
- 聚类 II: 纯度 = $4/6$
- 聚类 III: 纯度 = $3/5$

Rand 指数

Rand 指数通过比较两个聚类划分（即两个不同的平面聚类）来计算正确分类的比例。

设 $U = \{u_1, \dots, u_R\}$ 和 $V = \{v_1, \dots, v_C\}$ 是 N 个数据点的两种不同划分，即两种不同的（平面）聚类。

- TP : 在两个聚类划分中都在同一簇中的点对数量（真正例）。
- TN : 在两个聚类划分中都不在同一簇中的点对数量（真负例）。
- FN : 在一个聚类划分中在同一簇但在另一个划分中不在同一簇中的点对数量（假负例）。
- FP : 在一个聚类划分中不在同一簇但在另一个划分中在同一簇中的点对数量（假正例）。

其 Rand 指数：

$$R \triangleq \frac{TP + TN}{TP + FP + FN + TN}$$

Rand 指数

在提供的示例中，有三个簇，每个簇包含不同数量的点。



$$TP + FP = \binom{6}{2} + \binom{6}{2} + \binom{5}{2} = 40$$

$$TP = \binom{5}{2} + \binom{4}{2} + \binom{3}{2} + \binom{2}{2} = 20$$

$$FP = TP + FP - TP = 40 - 20 = 20$$

$$FN = 24$$

$$TN = 72$$

$$TP + TN$$

$$20 + 72$$

令 $p_{UV}(i, j) = \frac{|u_i \cap v_j|}{N}$ 为随机选择的对象属于 U 中簇 u_i 和 V 中簇 v_j 的概率。同样，令 $p_U(i) = \frac{|u_i|}{N}$ 为随机选择的对象属于 U 中簇 u_i 的概率；定义 $p_V(j) = \frac{|v_j|}{N}$ 类似。

$$\mathbb{I}(U, V) = \sum_{i=1}^R \sum_{j=1}^C p_{UV}(i, j) \log \frac{p_{UV}(i, j)}{p_U(i)p_V(j)}$$

归一化互信息

$$NMI(U, V) \triangleq \frac{\mathbb{I}(U, V)}{(\mathbb{H}(U) + \mathbb{H}(V))/2}$$

其他分区方法

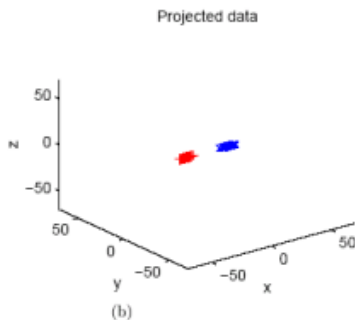
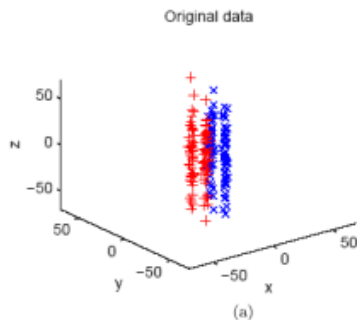
- 围绕中心点的划分 (PAM): 不用平均值, 而用多维中位数作为质心 (聚类“原型”)
- 自组织映射 (SOM): 添加底层“拓扑” (晶格上的邻接结构), 将聚类质心相互关联
- 模糊 K 均值: 允许点在不同聚类之间“渐变”; 软划分
- 基于混合模型的聚类: 通过 EM (期望最大化) 算法实现。提供软划分, 并允许建模聚类质心和形状

Outline

- 1 聚类理论
 - 聚类的定义
 - 距离度量
- 2 聚类算法
 - 层次算法
 - 分区算法
 - K-means 算法
 - 聚类的影响因素
- 3 补充学习

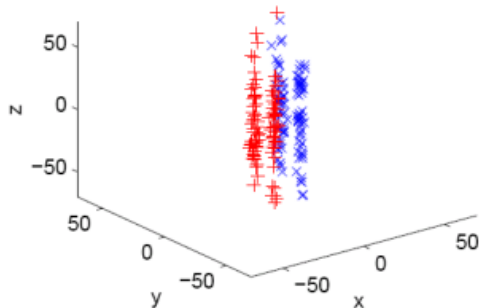
半监督度量学习

- 半监督度量学习 (Semi-Supervised Metric Learning, SSDML) 是一种结合少量标记样本和大量未标记样本来学习度量的方法。
- SSDML 尝试利用有限的标记数据和丰富的未标记数据来学习一个区分性嵌入空间，其中相似的样本更接近，而不相似的样本更远离



好的度量

- 在输入空间上，什么是一个好的度量，以及它对于学习和数据挖掘的意义是什么。



- 如何将人类用户有意义的度量（例如，沿高速公路车道而非立交桥划分交通，根据写作风格而非主题分类文档）系统地传达给计算机数据挖掘者？

学习度量中的问题

- 数据分布是自信息的（例如，位于子流形中）
通过找到数据在某个空间中的嵌入来学习度量（缺点：不反映（改变）人类的主观性。）
- 显式标记的数据集为关键特征提供线索
监督学习（缺点：需要大量同构训练）
- 侧面信息（例如， x 和 y 看起来（或读起来）相似）
提供少量定性和较少结构的侧面信息通常比明确陈述度量（写作风格的度量应该是什么？）或标记大量训练集容易得多
- 学习更有信息量的度量
我们能否使用少量侧面信息学习比欧几里得距离更有信息量的距离度量？

距离度量学习

侧面信息：假设对于某些点集 $\{c_i\}_{i=1}^R$ ，我们给定：

- S ：如果 c_i 和 c_j 相似，则 $(c_i, c_j) \in S$
- D ：如果 c_i 和 c_j 不相似，则 $(c_i, c_j) \in D$

距离度量学习：学习一个距离度量 $d(x, y)$ ，使得相似的点对（属于 S ）之间的距离较小。这里，距离度量定义为：

$$d(x, y) = d_A(x, y) = \|x - y\|_A = \sqrt{(x - y)^T A (x - y)}$$

其中， A 是一个参数矩阵，参数化了 $\mathbb{R}^n$ 上的一族马氏距离。学习 A 等价于找到数据的重新缩放方式： $x \mapsto A^{1/2}x$

最优距离度量

- 学习相对于侧面信息的最优距离度量导致以下优化问题：

$$\begin{aligned} \min_A \quad & \sum_{(i,j) \in S} \|c_i - c_j\|_A^2 \\ \text{s.t.} \quad & \sum_{(i,j) \in D} \|c_i - c_j\|_A^2 \geq 1, \quad A \succ 0 \end{aligned}$$

目标是最小化相似点对之间的距离平方和，同时确保不相似点对之间的距离平方和至少为 1，且 A 是正定的。

- 这个优化问题是凸的。这意味着存在没有局部最小值的算法，可以找到全局最优解。
- Xing 等人 2003 提供了一个高效的梯度下降 + 迭代约束投影方法来解决这个问题。

学习距离度量的示例

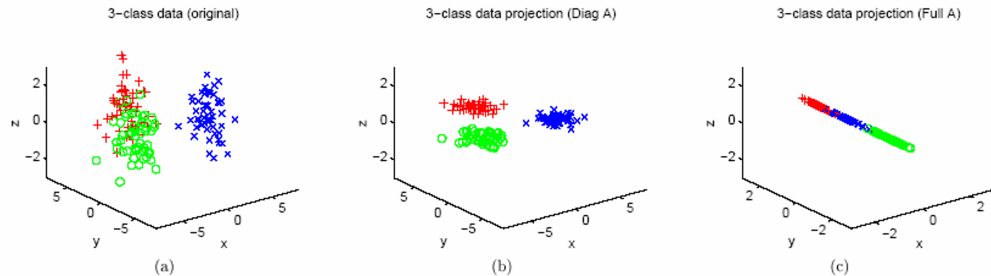


Figure 2: (a) Original data. (b) Rescaling corresponding to learned diagonal A . (c) Rescaling corresponding to full A .

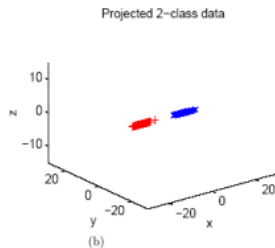
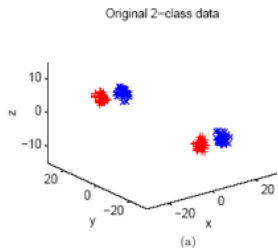
- (a) 原始数据 (Original data)
- (b) 应用了对角矩阵 A 进行重新缩放后的数据投影
- (c) 应用了完整的矩阵 A 进行重新缩放后的数据投影

改进的 K-means

- 约束 K-means (Constrained K-Means) : 引入额外的约束条件来优化聚类结果。
可以在聚类优化问题中显式添加约束, 要求每个聚类至少包含一定数量的点 (记为 γ_h)
- K-means+: 优化初始聚类中心的选择来提高聚类质量。
在初始化阶段, 根据数据点的距离来选择初始聚类中心, 使得初始聚类中心之间的距离尽可能远, 从而提高聚类质量
- 约束 K-means+
适合于那些需要考虑特定约束条件的聚类任务, 例如在社交网络分析、生物信息学和图像分割等领域。

聚类的应用

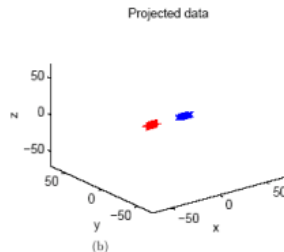
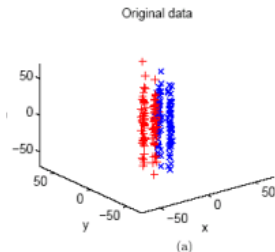
人工数据 I：困难的两类数据集



- ① K-means: Accuracy = 0.4975
- ② 约束 K-means: Accuracy = 0.5060
- ③ K-means + : Accuracy = 1
- ④ 约束 K-means + : Accuracy = 1

聚类的应用

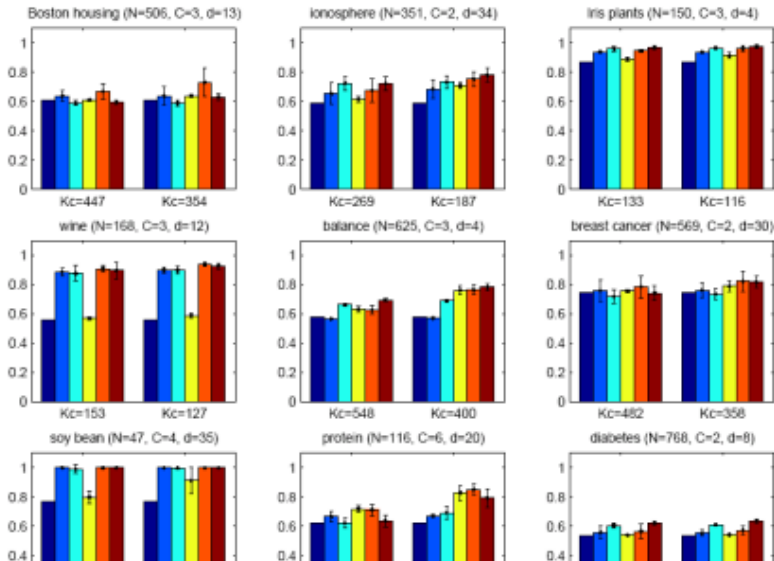
人工数据 II：具有强无关特征的两类数据



- ① K-means: Accuracy = 0.4993
- ② 约束 K-means: Accuracy = 0.5701
- ③ K-means + : Accuracy = 1
- ④ 约束 K-means + : Accuracy = 1

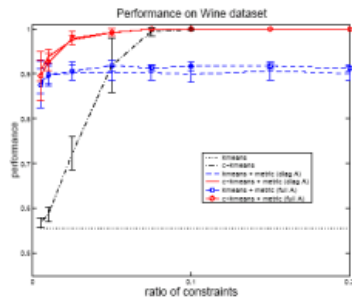
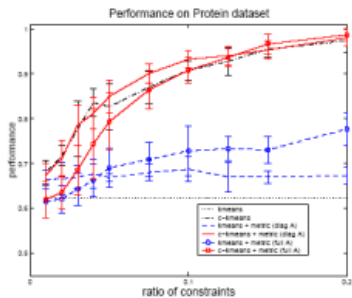
聚类的应用

来自 UC Irvine 仓库的 9 个数据集



准确率与侧面信息量

两个典型的例子，说明发现的聚类的质量如何随着侧信息量的增加而提高。



- 距离度量学习是机器学习和数据挖掘中的一个关键问题，它对于提高模型的性能至关重要。
- 通过解决凸优化问题，我们可以从少量的侧面信息中学习到有效的距离度量，这些信息通常以相似性和不相似性约束的形式给出。
- 学习到的距离度量能够识别特征空间中最能分离数据的重要方向，从而实现有效的隐式特征选择，这有助于提高模型的泛化能力。
- 此外，这些学习到的距离度量还可以用于改进聚类算法，使得聚类结果更加符合数据的内在结构和分布。

Feedback welcome!

The End